

UNIVERSITY OF KARACHI

UBIT — Department of Computer Science

ResumeFit

An AI-Assisted ATS Resume Builder and Scorer

A MERN Stack SaaS Application Documentation

Submitted By

Name	Seat Number
Ayesha Ahmed	B22110006032
Khola Moiez	B22110006068
Pooja Deep	B22110006130

Table of Contents

1. Abstract	3
2. Introduction.....	3
3. Objectives.....	3
4. Technology Stack.....	4
5. System Architecture.....	4
6. Database Design.....	5
7. Module Description.....	6
8. API Reference	7
9. Implementation Highlights.....	7
10. Testing	8
11. Conclusion	9
12. Future Scope	9
13. References.....	9

1. Abstract

Applicant Tracking Systems (ATS) are used by the majority of modern recruiters to filter resumes before a human ever reviews them, most commonly by scanning for keyword overlap with the job description. Many qualified candidates are rejected at this automated stage simply because their resume is phrased differently from the posting, not because they lack the underlying skills. ResumeFit addresses this problem directly: it lets a user upload or write a resume, paste in a target job description, and receive an instant, transparent match score along with the exact keywords the resume is missing.

The system is built as a complete, multi-tenant SaaS product using the MERN stack (MongoDB, Express.js, React, Node.js). Beyond the core scoring feature, it supports user authentication, per-user resume management with file upload (PDF/DOCX) and text extraction, team workspaces for organisations such as career-counselling offices, and a billing module ready to accept subscription payments through Stripe. This document describes the problem, objectives, system design, technology stack, module breakdown, and implementation details of the project.

2. Introduction

2.1 Background

Job seekers frequently tailor a single resume to dozens of postings without visibility into how an ATS actually parses and ranks their document. Existing solutions in the market are either paid, closed-source, or give vague, unexplained scores. There is a clear need for a tool that is transparent about how the score is computed and that scales from an individual job seeker to a team helping many candidates at once.

2.2 Problem Statement

Job seekers lack an accessible, explainable way to check whether their resume contains the keywords an ATS is likely to weight for a given job posting, and career-services teams lack a shared workspace to do this at scale for the people they support.

2.3 Scope of the Project

- User registration and authentication with JSON Web Tokens (JWT)
- Resume creation, editing, deletion, and file upload (PDF / DOCX) with automatic text extraction
- A keyword-based ATS scoring engine that compares a resume against a pasted job description
- Multi-user workspaces (organisations) with an invite-code join flow and role-based access (owner, admin, member)
- A billing module integrated with Stripe Checkout and webhooks for subscription management
- A responsive React front-end covering the full user journey from landing page to dashboard

3. Objectives

- Design and implement a transparent, explainable ATS-style resume scoring algorithm
- Build a secure, role-based multi-tenant backend using Node.js, Express, and MongoDB
- Implement resume file upload with server-side text extraction from PDF and DOCX formats
- Provide a clean, distinctive front-end user experience using React and Tailwind CSS
- Integrate a real-world payment gateway (Stripe) to demonstrate a production-ready billing flow
- Apply MERN stack best practices: RESTful API design, JWT-based authentication, and component-based front-end architecture

4. Technology Stack

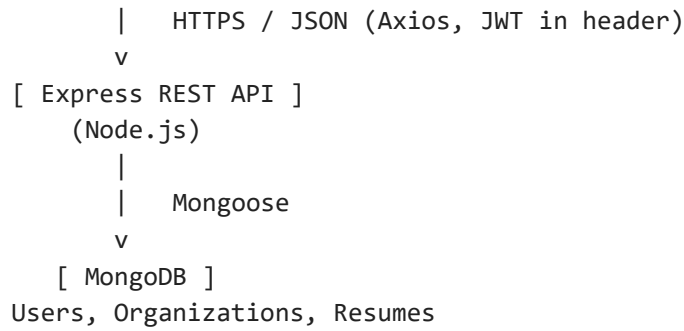
Layer	Technology	Purpose
Database	MongoDB (Mongoose ODM)	Stores users, organizations, and resumes
Backend runtime	Node.js	JavaScript runtime for the API server
Backend framework	Express.js	REST API routing and middleware
Authentication	JSON Web Tokens (JWT), bcrypt.js	Stateless auth and password hashing
File handling	Multer, pdf-parse, mammoth	Resume upload and text extraction (PDF/DOCX)
Payments	Stripe API	Subscription checkout and webhook handling
Frontend library	React 18	Component-based user interface
Build tool	Vite	Front-end dev server and bundler
Routing	React Router	Client-side navigation
Styling	Tailwind CSS	Utility-first responsive styling
HTTP client	Axios	Communication between front-end and API

5. System Architecture

ResumeFit follows a standard three-tier MERN architecture: a React single-page application communicates with an Express REST API over HTTP, and the API persists data in MongoDB. Authentication is stateless, using JWTs issued at login/registration and validated by middleware on every protected route.

5.1 High-Level Architecture Diagram

```
[ React Frontend ]
  (Vite, Tailwind)
      |
```



External services: Stripe (billing) · pdf-parse / mammoth (resume file parsing)

5.2 Request Flow (ATS Scan Example)

- User selects a saved resume and pastes a job description in the browser
- React sends a POST request to `/api/ats/score` with the resume ID and job text, including the JWT in the Authorization header
- Express middleware verifies the JWT and attaches the authenticated user to the request
- The controller loads the resume from MongoDB and passes both texts to the scoring engine
- The scoring engine tokenises both texts, extracts the job description's top keywords, and checks which are present in the resume
- The result (match percentage, matched keywords, missing keywords) is saved back onto the resume document and returned to the client
- React renders the score and keyword breakdown

6. Database Design

The database uses three core Mongoose collections.

6.1 User

Field	Type	Notes
name	String	Required
email	String	Required, unique, lowercase
password	String	Hashed with bcrypt before save
role	String	owner / admin / member
organization	ObjectId	Reference to Organization

6.2 Organization

Field	Type	Notes
name	String	Workspace / team name

Field	Type	Notes
plan	String	free / pro
seatsUsed	Number	Count of members
stripeCustomerId	String	Set once billing is activated
stripeSubscriptionId	String	Set once billing is activated
inviteCode	String	Unique code used to join the workspace

6.3 Resume

Field	Type	Notes
owner	ObjectId	Reference to User
organization	ObjectId	Reference to Organization
title	String	Resume title / file name
rawText	String	Full extracted or typed resume content
fileName / fileType	String	Set when uploaded from a file
lastScore	Object	matchPercent, matchedKeywords, missingKeywords, scoredAt

7. Module Description

7.1 Authentication Module

Handles registration (which also creates a new workspace), login, joining an existing workspace via invite code, and fetching the current session's user and organisation. Passwords are hashed with bcrypt; sessions are JWT-based with a 30-day expiry.

7.2 Resume Management Module

Allows a user to list, create, edit, and delete resumes, and to upload a PDF or DOCX file. Uploaded files are parsed server-side (pdf-parse for PDFs, mammoth for DOCX) to extract plain text, which is what the scoring engine actually evaluates — mirroring how a real ATS ingests a document.

7.3 ATS Scoring Module

The core feature. Given a resume and a job description, the engine tokenises both, strips common stop-words, ranks the job description's most frequent terms as its keyword set, and checks which of those keywords appear in the resume. It returns a match percentage plus explicit matched and missing keyword lists, and persists the result on the resume record.

7.4 Team / Workspace Module

Every account belongs to an organisation (workspace). Owners and admins can view all members and remove them; the workspace has a shareable invite code so a career-services team can bring multiple counsellors or students into one shared space.

7.5 Billing Module

Implements a Free and a Pro plan. The Pro upgrade path creates a Stripe Checkout session server-side; a webhook endpoint listens for successful payments and subscription cancellations to keep the workspace's plan in sync with Stripe. The module detects whether Stripe keys are configured and disables itself gracefully if not, so the rest of the application still runs without a payment provider.

8. API Reference

Method	Endpoint	Description
POST	/api/auth/register	Create a user and a new workspace
POST	/api/auth/login	Authenticate and receive a JWT
POST	/api/auth/join	Join an existing workspace via invite code
GET	/api/auth/me	Get the current user and workspace
GET	/api/resumes	List the current user's resumes
POST	/api/resumes	Create a blank resume
POST	/api/resumes/upload	Upload a PDF/DOCX resume
PUT	/api/resumes/:id	Update a resume's title or content
DELETE	/api/resumes/:id	Delete a resume
POST	/api/ats/score	Score a resume against a job description
GET	/api/team	List workspace members
DELETE	/api/team/:userId	Remove a workspace member
GET	/api/billing/status	Get the workspace's current plan
POST	/api/billing/checkout	Create a Stripe Checkout session
POST	/api/billing/webhook	Receive Stripe billing events

9. Implementation Highlights

9.1 ATS Scoring Algorithm

The scorer avoids depending on a paid external AI API so the project runs fully offline. It works in three steps:

- Tokenise: lowercase the text, strip punctuation, and remove a curated stop-word list
- Rank: count word frequency in the job description and take the top N terms as its 'keyword set'
- Match: check which of those keywords appear anywhere in the resume's tokenised text, and compute $\text{matchedCount} \div \text{totalKeywords}$ as a percentage

9.2 Security Considerations

- Passwords are never stored in plain text — bcrypt hashing with a salt round of 10
- All resume and team routes require a valid JWT, verified by Express middleware
- Resumes are always scoped to `req.user._id`, so one user cannot read or modify another user's data
- Stripe webhook signatures are verified before any billing state is trusted

9.3 Frontend Architecture

The React app uses a central AuthContext to hold the logged-in user and expose login/register/logout to any component. Protected routes (dashboard, resume editor, ATS checker, team, billing) redirect to the login page if no valid session exists. Axios is configured with an interceptor that automatically attaches the stored JWT to every request.

10. Testing

Test Case	Input	Expected Result	Status
User registration	Valid name, email, password	Account and workspace created, JWT returned	Pass
Duplicate email registration	Already-registered email	409 error with clear message	Pass
Login with wrong password	Valid email, incorrect password	401 Unauthorized	Pass
Resume upload (PDF)	Sample .pdf resume	Text extracted and resume record created	Pass
Resume upload (DOCX)	Sample .docx resume	Text extracted and resume record created	Pass
ATS scoring	Resume + job description	Match percentage and keyword lists returned	Pass
Unauthorized access	Protected route, no token	401 Unauthorized	Pass
Team member removal	Non-owner attempts removal	403 Forbidden	Pass

11. Conclusion

ResumeFit demonstrates a complete, production-shaped SaaS application built on the MERN stack, covering authentication, file processing, a custom-built scoring algorithm, multi-tenant workspaces, and payment integration. The project satisfies the core objective of giving job seekers a transparent, actionable view of how their resume compares to a job description, while also showing the team's ability to design and implement a full-stack system end to end.

12. Future Scope

- Replace the keyword-frequency scorer with an embedding-based semantic similarity model for more nuanced matching
- Add a structured resume builder (separate fields for experience, education, skills) instead of a single free-text block
- Add PDF export of the edited resume
- Add email verification and password-reset flows
- Add rate limiting and analytics on the scoring endpoint for production hardening